ASSIGNMENT-3.4

NAME:-PRATHAMKOUSHIK

ROLLNO:-2403A52182

Task 1: Zero-shot Prompt – Fibonacci Series Generator

Task Description #1

• Without giving an example, write a single comment prompt asking GitHub Copilot to
generate a Python function to print the first N Fibonacci numbers.

Expected Output #1

• A complete Python function generated by Copilot without any example provided.

• Correct output for sample input N = 7 → 0 1 1 2 3 5 8

• Observation on how Copilot understood the instruction with zero context

```
[1]  def generate_fibonacci(n):
       """Generates the first n Fibonacci numbers."""
       fibonacci_sequence = []
       a, b = 0, 1
       for _ in range(n):
         fibonacci_sequence.append(a)
         a, b = b, a + b
       return fibonacci_sequence

     # Example usage:
     n = 10
     fib_numbers = generate_fibonacci(n)
     print(f"The first {n} Fibonacci numbers are: {fib_nu

     The first 10 Fibonacci numbers are: [0, 1, 1, 2, 3,
```

Task 2: One-shot Prompt – List Reversal Function

Task Description #2

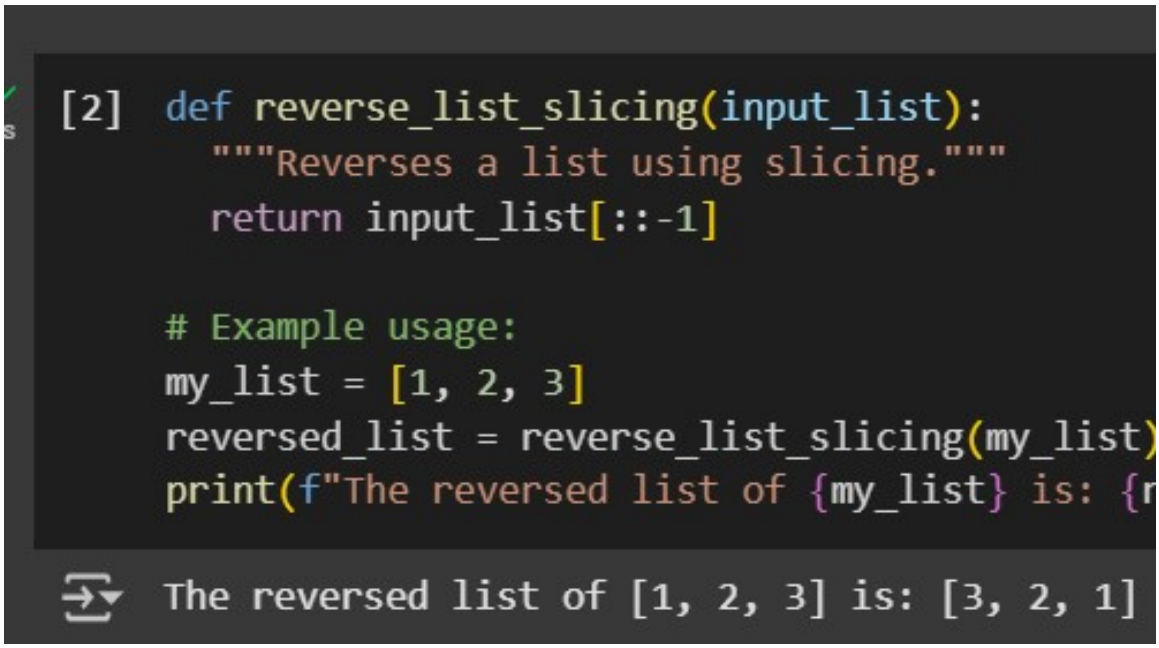
• Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

• Copilot-generated function to reverse a list using slicing or loop.

• Output: [3, 2, 1] for input [1, 2, 3]

• Observation on how adding a single example improved Copilot's accuracy



Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

• Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

• A function is_valid() that checks the pattern.

• Output: True or False based on input.

• Students reflect on how multiple examples guide Copilot to generate more accurate code.

```
[3]  def is_valid(input_string):
       """
       Checks if a string starts with a capital letter a
       """
       if not input_string:
         return False
       return input_string[0].isupper() and input_string

     # Examples:
     print(f'is_valid("Hello world.") -> {is_valid("Hell
     print(f'is_valid("hello world.") -> {is_valid("hell
     print(f'is_valid("Hello world") -> {is_valid("Hello

     is_valid("Hello world.") -> True
     is_valid("hello world.") -> False
     is_valid("Hello world") -> False
```

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

• First, prompt Copilot to write an email validation function using zero-shot (just the
task in comment).

• Then, rewrite the prompt using few-shot examples.

Expected Output #4

• Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

• Submit both code versions and note how few-shot improves reliability

```
[5]  import re

     def is_valid_email(email_string):
       """
       Validates if a given string is a valid email address.
       """
       # A basic regex for email validation. More complex regex can be use
       email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
       if re.match(email_regex, email_string):
         return True
       else:
         return False

     # Examples:
     print(f'is_valid_email("test@example.com") -> {is_valid_email("test@e
     print(f'is_valid_email("user.name@domain.co") -> {is_valid_email("use
     print(f'is_valid_email("username@") -> {is_valid_email("username@")}'
     print(f'is_valid_email("username.domain.com") -> {is_valid_email("use

     is_valid_email("test@example.com") -> True
     is_valid_email("user.name@domain.co") -> True
     is_valid_email("username@") -> False
     is_valid_email("username.domain.com") -> False
```

Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

• Experiment with 2 different prompt styles to generate a function that returns the sum
of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example

Expected Output #5

• Two versions of the sum_of_digits() function.

• Example Output: sum_of_digits(123) → 6

• Short analysis: which prompt produced cleaner or more optimized code and why

```
[6]  def sum_digits(number):
       """
       Calculates the sum of the digits of a number.
       """
       sum_of_digits = 0
       for digit in str(number):
         sum_of_digits += int(digit)
       return sum_of_digits

     # Example usage:
     num = 12345
     digit_sum = sum_digits(num)
     print(f"The sum of the digits of {num} is: {digit_su

     num = 987
     digit_sum = sum_digits(num)
     print(f"The sum of the digits of {num} is: {digit_su
```

```
The sum of the digits of 12345 is: 15
The sum of the digits of 987 is: 24
```

Gemini

```
[7]  def sum_digits(number):
       """
       Calculates the sum of the digits of a numbe
       """
       sum_of_digits = 0
       for digit in str(number):
         sum_of_digits += int(digit)
       return sum_of_digits

     # Example usage:
     print(f"sum_of_digits(123) -> {sum_digits(123)
```

```
sum_of_digits(123) -> 6
```